Chapter 3 Machine Learning Workflows

Austin R.J. Downey

Machine Learning Workflows

A machine learning workflow transforms raw data into a trained and evaluated model.

Important steps:

- Collect data
- Inspect and clean data
- Engineer features
- Train and test model
- Improve model

The workflow is almost always iterative.

Typical machine learning workflow.

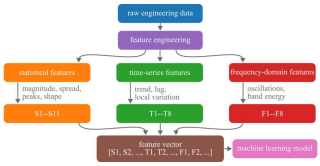
Feature Engineering

Feature engineering transforms raw data into useful model inputs.

Good features include quantities, categorical, or countable.

They can also be extracted from signals:

- magnitude
- variability
- time-dependent behavior
- frequency content
- physical relationships



Transforming raw data into a feature vector.

Statistical Features

Statistical features summarize the distribution of values within a dataset, observation, or time window.

No.	Feature	Interpretation
S1	Maximum value	Largest response in the signal.
S2	Minimum value	Smallest response in the signal.
S3	Mean value	Average level of the signal.
S4	Absolute mean value	Average magnitude of the signal.
S5	Root mean square value	Effective signal magnitude.
S6	Standard deviation	Variation in the signal.
S7	Skewness	Bias toward one side.
S8	Kurtosis	Sharp peaks or outliers.
S9	Shape factor	Effective magnitude relative to average magnitude.
S10	Crest factor	Largest peak relative to effective level.
S11	Impulse factor	Largest peak relative to average magnitude.

* The full equations are provided in Table 1 of Chapter 3 in the accompanying text.

Time-Series Features

Time-series features describe how a signal changes with time.

No.	Feature	Interpretation
T1	Lagged value	Recent history of the signal.
T2	First difference	Step-to-step change in the signal.
T3	Rate of change	How quickly the signal changes with time.
T4	Moving average	Local trend of the signal.
T5	Rolling standard deviation	Local variability of the signal.
T6	Rolling maximum	Recent peak behavior.
T7	Rolling minimum	Recent low-response behavior.
T8	Zero-crossing rate	How often the signal changes sign.

* The full equations are provided in Table 2 of Chapter 3 in the accompanying text.

Frequency-Domain Features

Frequency-domain features describe how signal content is distributed across frequency.

No.	Feature	Interpretation
F1	Dominant frequency	Strongest periodic behavior.
F2	Spectral centroid	Center of the spectrum.
F3	Spectral bandwidth	Spread of energy across frequency.
F4	Low-frequency energy ratio	Share of energy in slow behavior.
F5	Middle-frequency energy ratio	Share of energy in mid-range behavior.
F6	High-frequency energy ratio	Share of energy in rapid behavior.
F7	High-to-low frequency energy ratio	Rapid behavior relative to slow behavior.
F8	Peak-to-total spectral energy ratio	Dominance of the largest frequency peak.

* The full equations are provided in Table 3 of Chapter 3 in the accompanying text.

Overfitting

Training and testing on the same dataset is a major mistake.

- The model can memorize the training data
- Training accuracy may appear perfect
- Performance on new data may be poor

This problem is called:

Overfitting

Goal:

- Build models that generalize to unseen data

Training and Testing Sets

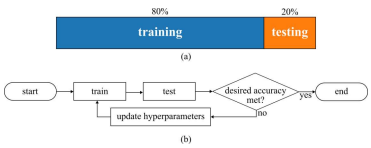
Standard supervised learning workflow:

- Train model using training data
- Evaluate model using test data
- Test data must remain unseen during training

Notation:

$$\mathbf{X}_{\text{train}}, \quad \mathbf{y}_{\text{train}}$$

$$\mathbf{X}_{\text{test}}, \quad \mathbf{y}_{\text{test}}$$



Training/testing split.

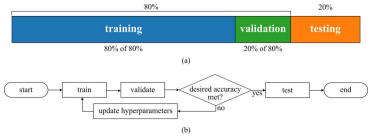
Validation Sets

A validation set is often added to tune models.

- Training set:
  - Learn model parameters
- Validation set:
  - Tune hyperparameters
- Test set:
  - Final evaluation

Scikit-Learn:

```
train_test_split
```



Training, validation, and testing workflow.

Machine Learning Pipelines

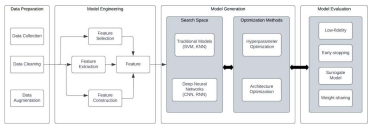
Scikit-Learn pipelines combine multiple processing steps into a single workflow.

Typical steps:

- Data preprocessing
- Feature scaling
- Feature engineering
- Model training

Benefits:

- Cleaner code
- Consistent preprocessing
- Easier cross-validation
- Simplified hyperparameter tuning



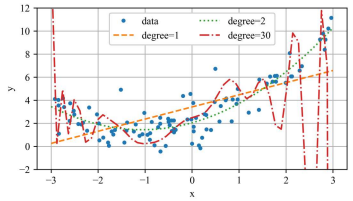
Example machine learning pipeline.

Learning Curves

Model complexity strongly affects performance.

- Low complexity:
  - Underfitting
- High complexity:
  - Overfitting
- Intermediate complexity:
  - Better generalization

Learning curves help diagnose model behavior.



Underfitting, good fit, and overfitting.

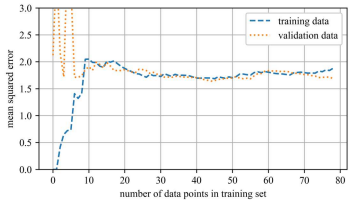
Underfitting

Characteristics of underfitting:

- Model is too simple
- Cannot capture data complexity
- Training and validation errors remain high

Learning curves:

- Curves are close together
- Errors plateau at high values



Learning curves for an underfitting model.

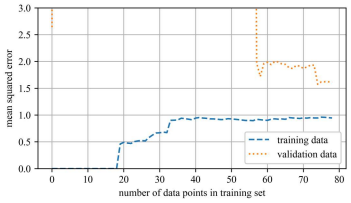
Overfitting

Characteristics of overfitting:

- Model is too complex
- Fits noise in the training data
- Excellent training performance
- Poor validation performance

Learning curves:

- Large gap between curves
- High variance



Learning curves for an overfitting model.

Well-Fit Model

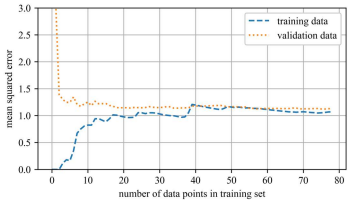
A good model balances:

- Bias
- Variance

Characteristics:

- Low training error
- Low validation error
- Small gap between curves

The model generalizes well to unseen data.



Learning curves for a well-fit model.

Interpreting Learning Curves

Learning curves compare:

Training Error vs. Validation Error

- Underfitting:
  - Both errors high
  - Curves close together
- Overfitting:
  - Training error low
  - Validation error much higher
- Good fit:
  - Both errors low
  - Curves converge

Example 3.1: Learning Curves

- Compare learning curves for:
  - Linear regression
  - Polynomial regression
- Plot training and validation errors as the training set grows
- Demonstrates:
  - Underfitting
  - Overfitting
  - Model generalization
  - Bias–variance tradeoff
- Shows how model complexity affects performance

Regularized Linear Models

Overfitting can be reduced through:

Regularization

Idea:

- Constrain model complexity
- Prevent fitting noise in the data
- Improve generalization

For polynomial models:

- Use a lower polynomial degree

For linear models:

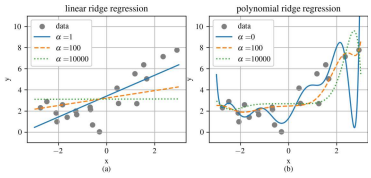
- Penalize large coefficient values

Ridge Regression

Ridge Regression adds a penalty term to the cost function:

$$J(\theta) = \text{MSE}(\theta) + \alpha \frac{1}{2} \sum_{i=1}^{n} \theta_i^2$$

- α: regularization strength
- Large $\alpha \rightarrow$ smaller weights
- Reduces overfitting



Ridge regression with different α values.

Note

The bias term θ_0 is not regularized.

Example 3.2: Ridge Regression

- Apply Ridge Regression to a high-degree polynomial model
- Add regularization to reduce overfitting
- Use a pipeline with:
  - `PolynomialFeatures`
  - `StandardScaler`
  - `Ridge`
- Demonstrates:
  - Smoother predictions
  - Reduced variance
  - Improved model stability
- Compare different values of α

Early Stopping

Early stopping is a form of regularization for iterative learning methods.

Idea:

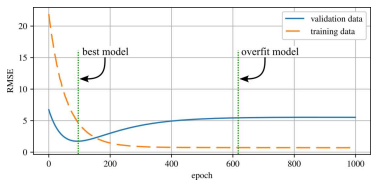
- Monitor validation error during training
- Stop training when validation error begins increasing

Benefits:

- Prevents overfitting
- Improves generalization
- Simple and computationally inexpensive

Key Idea

Stop training at the minimum validation error.



Early stopping prevents overfitting.

Example 3.3: Early Stopping

- Train a high-degree polynomial model using:
  - Stochastic Gradient Descent
- Track:
  - Training error
  - Validation error
- Demonstrates:
  - How overfitting develops during training
  - How validation error identifies the optimal stopping point
  - Improved generalization through early stopping
- Training halts once validation performance begins degrading